

PROBLEM STATEMENT: HTML/CSS/JS ASSESSMENT ENGINE USING PUPPETEER (PARTIAL MARKS + VISUAL DIFF MAP + EXTERNAL LIBRARIES)

1) Goal

Build an automated evaluation engine for **HTML/CSS/JS assessments** that runs student submissions in a **sandboxed Chromium environment via Puppeteer**, grades them with:

- **DOM + JS behavior tests**
 - **CSS computed-style tests**
 - **Visual snapshot comparison with diff heatmap (“visual map”)**
 - **Partial scoring** with rubric weights
 - Controlled **external library** support
-

2) Users

- **Student:** writes code, submits, sees score + visual map + feedback
 - **Trainer/Content team:** authors questions, defines tests/rubric, generates baseline
 - **Admin:** manages library policy, monitoring, evaluation capacity and logs
-

3) Scope

In Scope

- Evaluate single-page web tasks (HTML/CSS/JS)
- Headless browser evaluation using **Puppeteer**
- DOM assertions, interaction simulation, computed-style assertions
- Screenshot baselines + diff image generation
- Partial scoring + section breakdown
- External libraries: allowed via **whitelist / library picker**
- Admin tooling: question editor + test spec + baseline generation

- Store artifacts: expected/actual/diff screenshots, logs, results JSON

Out of Scope (Phase 1)

- Bundling for complex SPA (React/Vue build toolchain) — can be Phase 2
 - Backend API tasks requiring real network (unless explicitly enabled per question)
-

4) Key Requirements

4.1 Functional Requirements

FR-1: Question Authoring

- Create/update question with:
 - title, description, instructions, time limit
 - starter files: index.html, style.css, script.js (minimum)
 - allowed libraries policy (picker or whitelist)
 - evaluation spec (tests + rubric)
 - viewports list for visual tests (desktop/mobile)

FR-2: Baseline Generation

- Admin provides “reference solution” (or uses final answer)
- System runs Puppeteer to:
 - render at specified viewports
 - capture baseline screenshot(s)
 - store baseline artifacts and version
- Baseline versioning required (v1, v2...)

FR-3: Submission Packaging

- On submit, system builds an isolated runnable page:
 - inject student HTML/CSS/JS
 - inject selected libraries (controlled)
 - optionally apply “assessment harness” script (for freezing animations, collecting logs)
- Store attempt metadata (user, timestamp, duration, question id, library selection)

FR-4: Static Validation

- HTML parse/validate (basic)
- CSS parse/validate
- JS syntax check
- Optional lint rules:
 - HTMLHint, Stylelint, ESLint (configurable per course)
- Static checks contribute to partial marks (not always blocking)

FR-5: Runtime Evaluation via Puppeteer

- Run in isolated Chromium context:
 - open page
 - block network except allowed domains (libraries)
 - enforce timeouts and resource limits
- Collect:
 - console logs/errors
 - runtime errors
 - DOM snapshot (optional)
- Execute tests:
 1. **DOM Tests**
 2. **CSS computed style tests**
 3. **Interaction tests** (click/type/keypress/scroll)
 4. **Functional assertions** (text updates, node changes, class toggles)

FR-6: Visual Diff Map (Visual Tests)

- For each viewport:
 - capture student screenshot
 - compare to baseline using pixel-diff with tolerance
 - produce:
 - diff image (heatmap overlay)
 - diff percentage

- visual score
- Provide “visual map” to student with Expected / Actual / Diff tabs

FR-7: Partial Scoring

- Rubric buckets (default, configurable):
 - HTML (structure/semantics)
 - CSS (layout/styling computed)
 - JS (behavior/interactions)
 - Visual (screenshot diff)
 - Quality (lint/accessibility)
- Each test has:
 - weight (marks)
 - pass criteria
 - failure hint
- Score = sum of passed weights (or partial per metric for visual)

FR-8: Result Report

Student sees:

- total score + bucket breakdown
- failed tests list (with hints)
- console/runtime error summary
- visual diff maps per viewport

Admin sees:

- full logs, timings, worker traces, artifacts, and replay capability

5) External Libraries Policy

Library modes (must implement at least one in Phase 1):

1. Library Picker (recommended)

- platform-controlled list (Bootstrap, Tailwind, jQuery, etc.)
- fixed versions per question

- auto-injected into runner
- 2. **Whitelist URLs/Domains**
 - allow only specified CDN domains
- 3. **Self-hosted mirror (optional, Phase 2)**
 - mirror libs on your server for reliability

Network Enforcement

- Default: request interception in Puppeteer blocks all network except allowed domains
- Per-question toggle: allow fetch/XHR if the question needs it

6) Evaluation Spec Format (JSON stored per question)

Minimum schema that dev team should implement.

```
{
  "viewports": [
    { "name": "desktop", "width": 1366, "height": 768 },
    { "name": "mobile", "width": 390, "height": 844 }
  ],
  "rubric": {
    "html": 20,
    "css": 35,
    "js": 35,
    "visual": 10
  },
  "timeouts": {
    "pageLoadMs": 5000,
    "testMs": 8000,
    "totalMs": 15000
  },
}
```

```
"domTests": [  
  {  
    "id": "dom-1",  
    "weight": 5,  
    "selector": "#submitBtn",  
    "assert": "exists",  
    "failHint": "Add a button with id submitBtn."  
  },  
  {  
    "id": "dom-2",  
    "weight": 5,  
    "selector": ".card",  
    "assert": "count>=3",  
    "failHint": "Render at least 3 cards with class card."  
  }  
],  
"styleTests": [  
  {  
    "id": "css-1",  
    "weight": 10,  
    "selector": ".container",  
    "property": "display",  
    "expect": "flex",  
    "failHint": "Use flex layout for the container."  
  },  
  {  
    "id": "css-2",  
    "weight": 10,
```

```
"selector": ".container",
"metric": "alignCenterApprox",
"tolerancePx": 8,
"failHint": "Center-align items horizontally."
}
],
"interactionTests": [
{
  "id": "js-1",
  "weight": 15,
  "steps": [
    { "action": "type", "selector": "#taskInput", "value": "Buy milk" },
    { "action": "click", "selector": "#addBtn" }
  ],
  "assertions": [
    { "type": "textContains", "selector": ".task-list", "value": "Buy milk" },
    { "type": "countEquals", "selector": ".task-item", "value": 1 }
  ],
  "failHint": "Click Add should append the task into the list."
}
],
"visualTests": [
{
  "id": "vis-1",
  "weight": 10,
  "viewport": "desktop",
  "maxDiffPercent": 3.0,
  "failHint": "Layout or spacing differs. Check alignment and margins."
}
```

```
}  
]  
}
```

7) Puppeteer Execution Rules (Must)

- Run each evaluation in an **isolated context**
- Enforce:
 - maximum execution time (hard timeout)
 - memory/cpu limits at container/worker level
- Block:
 - popups, new windows
 - external network except allowed libraries
 - downloads
- Capture console and runtime errors

8) Visual Diff Implementation Requirements

- Screenshot capture rules:
 - fixed viewports
 - wait strategy: networkidle + small settle delay
 - disable animations/transitions (inject CSS * { animation: none; transition: none; })
- Diff:
 - generate diff image (Expected vs Actual)
 - calculate diff percentage
 - map diff% → score (configurable)
 - ≤1%: full marks
 - 1-3%: 80%
 - 3-6%: 50%

- 6%: 0

Artifacts stored:

- expected.png
- actual.png
- diff.png

9) Result & Feedback Requirements

Student Report UI

- Score summary + bucket-wise progress bars
- Failed tests list with:
 - test name
 - hint
 - optional “where to look” (selector)
- Visual diff section:
 - tabs: Expected / Yours / Diff
 - per viewport selection

Admin Report UI

- Full JSON evaluation report
- Console logs + stack traces
- Timing: load/test/screenshot durations
- Replay evaluation button (same submission)

10) Data Storage

Tables/Collections:

- questions
- question_files (starter + reference)
- test_specs (json)
- baselines (question_id, version, viewport, expected_image_path)

- submissions
 - evaluation_runs (submission_id, status, score, breakdown_json, logs_path)
 - artifacts (expected/actual/diff image paths)
-

11) APIs (Minimum)

- POST /questions (create)
 - POST /questions/{id}/baseline (generate baseline)
 - POST /submissions (submit attempt)
 - GET /submissions/{id}/result
 - GET /admin/evaluation_runs/{id}
-

12) Acceptance Criteria

1. For a question with DOM + style + interaction + visual tests:
 - evaluation returns total score + breakdown within configured timeouts
 2. Diff map images are generated and visible to student
 3. External libraries load only from allowed sources (others blocked)
 4. Same submission produces same result repeatedly (within tolerance)
 5. Infinite loop JS doesn't hang the system (job is killed, report shows timeout)
-

13) Phased Delivery

Phase 1 (MVP)

- Puppeteer runner + request interception
- DOM tests + interaction tests
- Visual diff for 1 viewport (desktop)
- Rubric-based partial scoring
- Student + Admin result pages

Phase 2

- Multiple viewports (desktop + mobile)

- Improved computed-style tests (bounding boxes/alignment tolerance)
- Baseline versioning + replay tool
- Library picker UI

Phase 3

- Component/region-based visual scoring (per section)
- SPA build pipeline support (React/Vue templates)
- More analytics (question difficulty, common failures)

14) Risks & Mitigations

- **Flaky visuals** (font/AA): fixed fonts or tolerance thresholds; disable animations
- **CDN issues**: cache libs or self-host mirror
- **Security**: strict network block + container isolation + timeouts
- **Scale**: queue-based workers; concurrent headless sessions capped per machine

We certified / Recognized by:

